

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Experimental design and provenance

The results in this chapter come from a single frozen state of the code base. Freezing the environment, the cost function and the calibrated thresholds before the first training run of the comparison, rather than after, is a deliberate part of the fairness protocol: it makes it impossible for a threshold to be adjusted in response to a result that has already been seen. All runs use the frozen weld environment `Isaac-Lift-Cube-UR5e-v0` described in Chapter 3.

Three arms are reported, as set out in Section 3.10.1. The `ctrl` arm carries the cost critic with its multiplier ceiling pinned to zero, so its policy update is stock PPO; it is labelled PPO (baseline) throughout.¹ The `cppo` arm is the constrained PPO-Lagrangian agent `[?, ?]` at an episodic cost budget of 25, the value inherited from the Safety Gym reference configuration. The `cppo15` arm is the same agent at a budget of 15, which is well below the baseline’s natural operating cost on every seed and is therefore actively binding rather than borderline.

Each arm was trained on five random seeds (1, 3, 4, 52 and 54) for 1500 iterations at 4096 parallel environments using `rsl_rl` 3.0.1, giving fifteen trained policies. All fifteen final checkpoints were verified present on disk rather than inferred from the absence of errors in the training logs. Evaluation was then carried out on the frozen, deterministic policy with observation corruption disabled, over three evaluation seeds (101, 102 and 103, disjoint from the training seeds) at 1000 episodes each. Every individual policy is therefore scored over 3000 episodes and every arm over 15,000, for 45,000 evaluation episodes in total. Reporting evaluation statistics from the frozen policy rather than from training-time telemetry is itself a correction carried forward from an earlier iteration of this work, in which safety percentages had been read from the TensorBoard scalars of a still-exploring policy and therefore could not describe the policy that would actually be deployed.

The safety thresholds were calibrated against a converged unconstrained baseline immediately before the freeze, as Section 3.9 records. Two points from that calibration matter for reading what follows. The manipulability floor was raised from 0.045 to 0.06 to restore the baseline violation rate to the intended band. The joint-limit margin was held at 0.175 rad

¹A plain ppo arm was trained on the same five seeds. Its stored weights proved byte-identical to `ctrl`’s, so `ctrl` stands in for it and the plain arm is not reported separately. The verification is Section 4.2; the raw comparison is `Comparison_test/ppo_redundant/README.md`.

but reclassified as active, because the baseline policy spends 33.7 % of its steps inside it, and it is the larger of the two active terms by realised cost. The collision floor of 0.05 m was confirmed inactive and remains so in every result below.

Table 4.1. Experimental configuration.

Item	Value
Task	Isaac-Lift-Cube-UR5e-v0 (frozen weld environment)
Arms reported	ctrl (PPO baseline), cppo ($d = 25$), cppo15 ($d = 15$)
Training seeds	1, 3, 4, 52, 54 ($n = 5$ per arm; 15 trained policies)
Training budget	1500 iterations, 4096 environments, rsl_rl 3.0.1
Evaluation seeds	101, 102, 103 ($n = 3$, disjoint from training)
Evaluation protocol	1000 episodes per checkpoint $\times$ evaluation seed, deterministic policy, observation corruption off
Evaluation episodes	3000 per policy, 15,000 per arm, 45,000 in total
MANIP_FLOOR	0.06 (recalibrated from 0.045)
JOINT_LIMIT_MARGIN	0.175 rad (active; 33.7 % baseline within-margin rate)
COLLISION_Z_FLOOR	0.05 m (confirmed inactive)
cost_limit	25 (cppo), 15 (cppo15)

4.2 Validity check: the control arm reproduces the baseline exactly

An earlier version of this comparison reported a large advantage for the constrained agent that was subsequently traced, in a line-by-line audit against the upstream library, to an implementation artifact rather than to the safety constraint. A single global gradient-norm clip had been applied across all parameters handed to the optimiser. Because the constrained agent carries a third network, the cost critic’s gradients entered that norm and scaled down the actor’s step on every update. The constrained arm was therefore not PPO plus a constraint, it was PPO with a systematically quieter optimiser, and a quieter optimiser is a well-known stabiliser on shaped-reward manipulation tasks. Because the multiplier had also sat at zero for essentially the whole of those runs, the constraint could not have been responsible for the gap at all. That result was withdrawn in full, the clip was partitioned so that the actor and reward critic receive treatment identical to the baseline, and the ctrl arm was added specifically so that the correction could be verified rather than assumed.

The verification is the strongest that this class of check admits, and it now rests on three independent comparisons.

First, the training scalars agree exactly. Across all five seeds, the tail-mean reward, the soft-margin singularity fraction and the minimum-manipulability trace all agree between ppo_sN and ctrl_sN to every decimal place the logs carry. The agreement extends to chaotic

near-zero quantities where any divergence in trajectory would be expected to show first: the minimum-manipulability tail mean reads 7.419×10^{-6} for both `ppo_s3` and `ctrl_s3`. Agreement at that precision is not what statistical equivalence looks like, and it was therefore treated as a suspected logging fault rather than as a result until it had been checked at the file level.

Second, the possibility that the two arms were reading the same log was excluded directly. The two runs wrote separate files, of different sizes, from separate processes, and the control arm’s file is the larger by an amount consistent with its additional cost-critic logging. The two final checkpoints were then opened and every stored tensor compared by content. All 68 tensors constituting `ppo_s1`’s trained actor and reward critic were found byte-for-byte inside `ctrl_s1`’s checkpoint, which carries 100 tensors in total, the remainder being the control arm’s own cost critic. Parameter names recovered from the serialised stream confirm that these are the weight and bias tensors of the four actor and four critic layers. Statistical indistinguishability would have been enough for the comparison. What the two arms in fact reached, at the level of the stored weights, is the same policy.

Third, the result reproduces at evaluation time on data the training runs never saw. Every one of the 15,000 evaluation episodes was compared episode by episode between the two arms on final goal distance, episode-minimum manipulability and episodic cost. All 15,000 agree to the precision the per-episode files carry, on all five seeds and all three evaluation seeds. Because the two arms are exactly equal wherever they are compared, they are reported as a single PPO (baseline) column throughout the rest of this chapter.

Interpretation requires one caveat that should be stated rather than glossed. With the clip partitioned and the multiplier pinned to zero, the control arm’s actor loss is algebraically identical to the baseline’s at every step, so identical *updates* are expected. Identical *weights* additionally require that the random-number stream driving action sampling and minibatch permutation was not displaced by the cost critic’s extra parameter draws at construction. The audit had assumed the opposite, and treated the resulting offset as unavoidable seed noise to be absorbed by the control arm. The empirical outcome contradicts that assumption. The effect is verified beyond reasonable doubt, but the mechanism, plausibly separate generator streams for host and device, was not traced in the library source, and is recorded here as an open question rather than asserted as an explanation. Nothing in the chapter depends on the explanation; the observation alone is what licenses the comparison that follows.

4.3 How this comparison must be read

The audit that withdrew the earlier result also fixed the form in which the corrected result may be reported. A direct constrained-versus-baseline number is not permitted as a headline, because such a number silently sums two effects of different kinds. What is reported instead

is the decomposition

$$(\text{cppo} - \text{ppo}) = (\text{ctrl} - \text{ppo}) + (\text{cppo} - \text{ctrl}) \quad (4.1)$$

in which the first term is the cost of merely attaching a cost critic, the implementation artifact, and the second is the effect of the constraint itself with everything else held fixed. Only the second term is a safe reinforcement learning result. The first term is the quantity that was mistakenly reported as the second in the withdrawn version of this work.

Section 4.2 establishes that the first term is exactly zero, on every seed and at every point of comparison. That is what makes the decomposition useful rather than merely cautious. Because the artifact term vanishes identically, the whole of any observed difference between a constrained agent and the baseline is attributable to the constraint, and the columns of every table below can be read directly. Had the first term been non-zero, no constrained-versus-baseline number could have been reported at all, because the arms would still have differed by something unnamed.

One further point governs the reading. The two constrained arms differ only in their budget, so the difference between them isolates the effect of *how hard* the constraint is applied, with the algorithm, the environment, the seeds and the network all held fixed. Section 4.8 treats that comparison on its own. The *sac* arm registered in the original plan was not trained, so no off-policy comparison is offered here.

4.4 Task performance

Table 4.2 reports training-time performance as a tail mean over the final tenth of training, with the spread across the five seeds.

Table 4.2. Training-time performance (tail mean over the final 10 % of iterations, mean $\pm$ sd across $n = 5$ seeds).

Quantity	Mean $\pm$ sd across $n = 5$ seeds		
	PPO (baseline)	cPPO, $d = 25$	cPPO, $d = 15$
Mean reward	133.06 $\pm$ 1.27	132.18 $\pm$ 1.99	134.09 $\pm$ 0.80
Mean episodic cost	81.86 $\pm$ 62.56	14.22 $\pm$ 4.38	10.94 $\pm$ 4.30
viol_singularity (step fraction)	0.455 $\pm$ 0.338	0.169 $\pm$ 0.043	0.233 $\pm$ 0.088
viol_joint_limit (step fraction)	0.076 $\pm$ 0.156	0.000	0.000

On reward the three arms are indistinguishable. The reward decomposition for the $d = 25$ arm is $(\text{cppo} - \text{ppo}) = 0.000 + (-0.880)$, so the entire difference of about 0.7 % arises from

the constraint and none from the artifact, and 0.880 is smaller than the seed-to-seed standard deviation of either arm. The tighter arm ends *above* the baseline at 134.09, and with the smallest spread of the three. Neither difference should be read as a measured task effect in either direction. What the row supports is that the constraint does not cost reward, and the honest form of that statement is an upper bound rather than a point estimate. Figure 4.1 shows the learning curves behind that row.

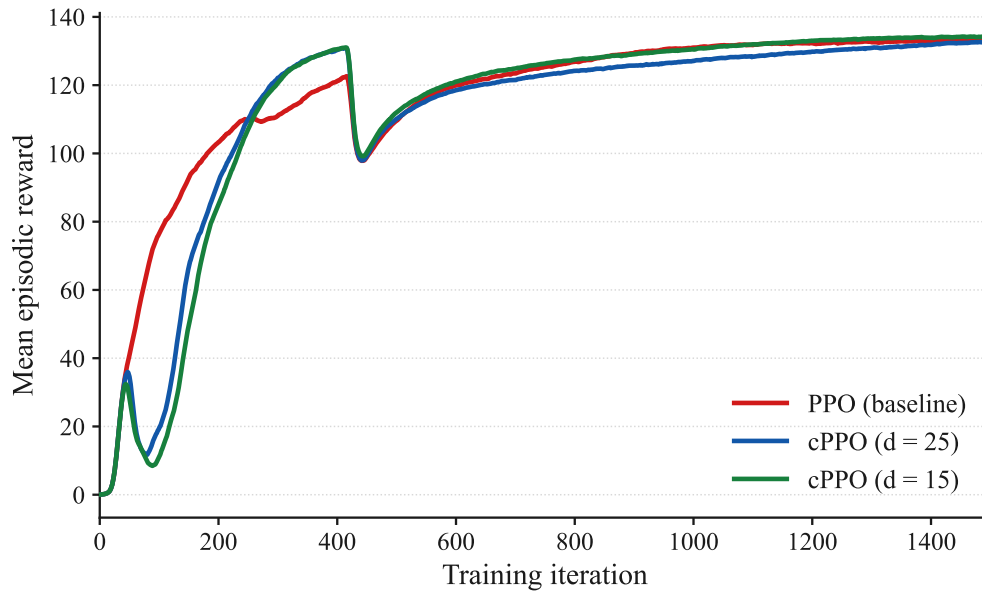


Figure 4.1. Mean episodic reward, averaged across the five seeds.

All three arms converge to the same level. The constrained arms learn more slowly for roughly the first two hundred iterations, which is the multiplier of Section 4.7 suppressing early exploration, and the gap closes well before training ends. Curves in this and every subsequent figure are exponentially smoothed for legibility; every value quoted in the text is computed from the unsmoothed logs.

The reward total is a sum of shaped terms, and the two that carry the task are plotted in Figure 4.2. Both converge to a common level on all three arms, so the equality of the reward row is not an artifact of one term compensating for another.

The soft-margin violation fractions in the lower rows are included for completeness and are deliberately not used as the safety headline. They count the fraction of control steps on which a continuous margin falls on the wrong side of a binary threshold, which exaggerates differences between policies whose margins differ only slightly, and they are measured on a still-exploring policy. Their dispersion illustrates the problem: the baseline’s standard deviation of 0.338 is twice the constrained agent’s mean. The rows are nevertheless informative in one respect that anticipates Section 4.6, in that the spread falls by roughly eightfold while the means separate by a factor of under three.

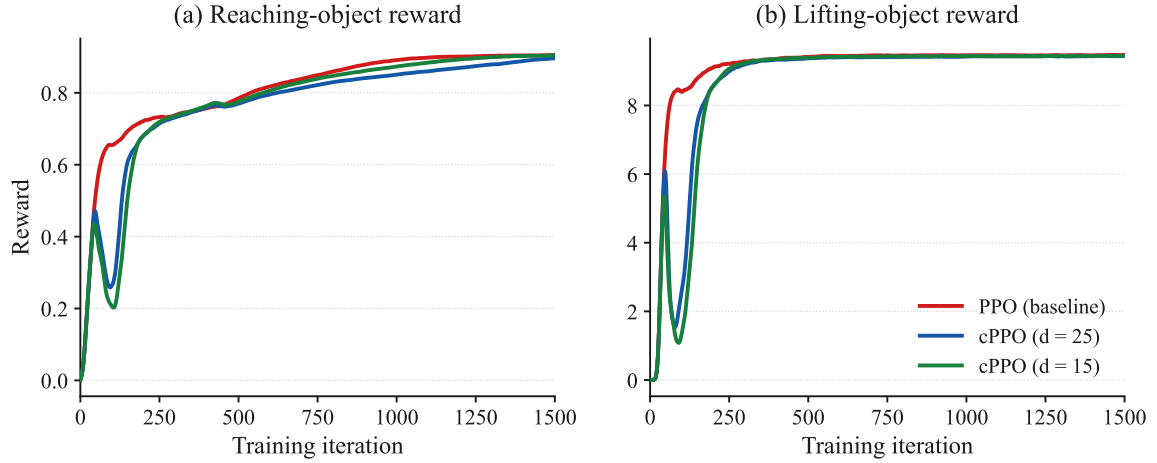


Figure 4.2. The two task-carrying reward terms, averaged across the five seeds. The arms are indistinguishable on lifting after roughly three hundred iterations, which is the training-time counterpart of the lift-success row of Table 4.3. Curves are exponentially smoothed for legibility.

Evaluation-time task performance is reported in Table 4.3. Following the audit’s reporting rule, goal-reach is given as a distance distribution together with success at three thresholds, never as a single figure. The reason is structural rather than statistical. Under the weld abstraction described in Section 3.5, the cube’s pose is the tool frame’s pose once the weld latches, so “cube within tolerance of the goal” reduces to “tool frame within tolerance of the goal”. A converged policy solves that on nearly every episode and a diverged one fails on nearly every episode, which drives any single-threshold success rate towards a ceiling and leaves it unable to rank two policies that have both converged. It was exactly this ceiling that produced the implausible 100 % against 0 % figures in the withdrawn version of this work.

Table 4.3. Evaluation-time task performance on the frozen deterministic policy. Each arm is 15,000 episodes (5 training seeds $\times$ 3 evaluation seeds $\times$ 1000). Bracketed figures are the standard deviation across the five training seeds.

Metric	PPO (baseline)	cPPO, $d = 25$	cPPO, $d = 15$
Lift success	99.91 % (0.09)	99.87 % (0.08)	99.90 % (0.08)
Goal-reach < 1 cm	93.03 % (7.52)	98.33 % (1.04)	98.93 % (0.95)
Goal-reach < 2 cm	98.78 %	99.58 %	99.84 %
Goal-reach < 5 cm	99.81 %	99.86 %	99.90 %
Goal distance, mean (m)	0.0062	0.0051	0.0047
Goal distance, median (m)	0.0050	0.0041	0.0041
Goal distance, p90 (m)	0.0087	0.0064	0.0059
Goal distance, max (m)	0.812	1.326	0.708

Every arm lifts the cube on essentially every episode, and the constrained arms are ahead of the baseline at every goal-reach threshold and across the whole distance distribution. The margins are small and should not be leaned on as evidence that the constraint improves the

task. The robust claim is the negative one, supported across seven independent measures: constraining the policy did not cost task performance, at either budget.

Figure 4.3 plots the success rows of that table twice. Panel (a) is the conventional view with the axis zoomed to the top of its range, since on a full scale from zero all twelve bars would be visually identical. Panel (b) plots the same numbers as failure rate on a logarithmic axis, where a difference between 99.91 and 93.03 % becomes a difference between 0.09 and 6.97 %, a factor of more than seventy. The second panel is the honest way to look at success rates this close to the ceiling, and it is the reason the goal-reach rows are reported at three thresholds rather than one.

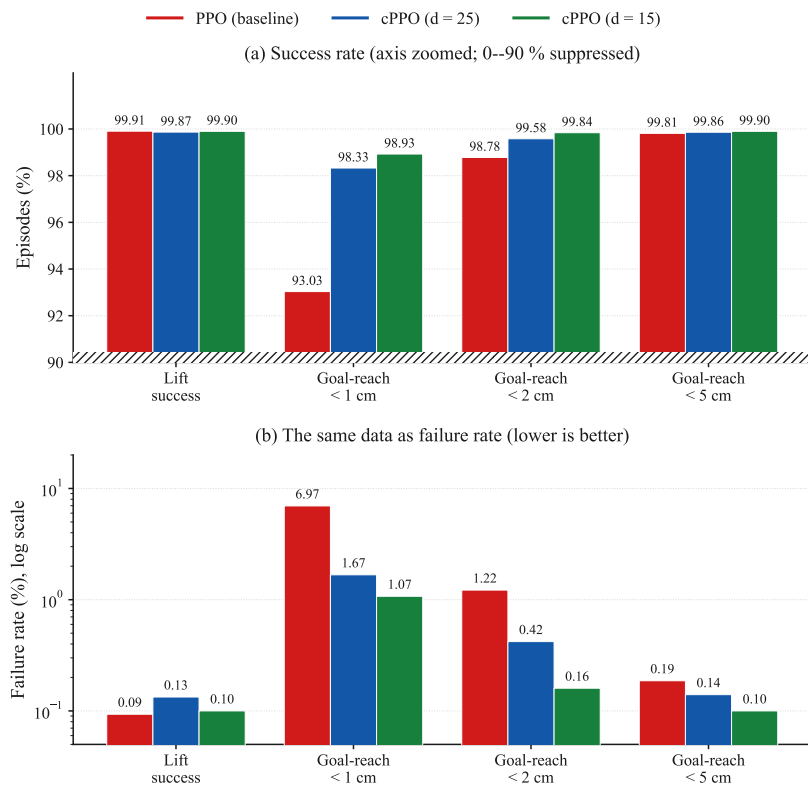


Figure 4.3. Evaluation-time task performance, 15,000 episodes per arm. Panel (a) shows success rates with the vertical axis zoomed to the 90 to 100 % band, the suppressed region marked by the hatched strip at the axis base. Panel (b) shows the complement, the failure rate, on a logarithmic axis, where differences invisible in panel (a) separate by more than a factor of seventy. Lower is better in panel (b).

One column of that table is more interesting than the margins, and it is the bracketed one. The seed-to-seed standard deviation of the sub-centimetre success rate falls from 7.52 percentage points on the baseline to 1.04 and 0.95 on the constrained arms, a sevenfold to eightfold tightening. The per-seed values behind the baseline figure run 80.00, 98.77, 94.83, 94.17 and 97.40 %, so one baseline seed in five lands roughly fifteen points below the others on task precision. Neither constrained arm has a seed below 96.97 %. The variance collapse that Section 4.6 reports for safety is therefore visible in task performance as well, which was not anticipated and is not something the constraint was designed to do.

Every arm produces a small number of catastrophic-miss episodes, with a maximum goal distance between 0.7 and 1.3 metres. These were not investigated and are recorded as an open item. They are noted here so that a reader encountering them in the per-episode data is not misled into treating them as an artifact of one arm.

4.5 Safety

Table 4.4 reports safety on the frozen policy, pooled over the 15,000 evaluation episodes per arm. Following the audit’s reporting rule, the table leads with episodes that reached an actual kinematic singularity, meaning a manipulability measure [?] below 10^{-4} , which is a statement about the arm’s genuine loss of a degree of freedom rather than about a calibrated soft margin.

Table 4.4. Safety on the frozen policy, pooled over 15,000 evaluation episodes per arm. Bracketed figures are the standard deviation across the five training seeds.

Metric	PPO (baseline)	cPPO, $d = 25$	cPPO, $d = 15$
True singularity crossings ($w < 10^{-4}$)	2.66 % (399) [3.79]	0.38 % (57) [0.36]	0.07 % (10) [0.13]
Mean episode-minimum w	0.0465	0.0669	0.0628
Worst single-episode w	0.000001	0.000001	0.000001
Joint limit touched at all	10.70 %	0.00 %	0.00 %
Collision touched at all	0.03 %	0.03 %	0.02 %
Episodic cost, mean	84.05	15.21	11.31
Episodic cost, p90	200.38	56.90	43.77
Episodic cost, max	343.01	219.59	228.51

The constraint reduces true singularity crossings by a factor of 7.0 at a budget of 25, from 399 episodes to 57 out of 15,000, and by a factor of 39.9 at a budget of 15, to 10 episodes. This is a stricter and more meaningful measurement than the soft-margin fraction of Table 4.2, and the effect survives the change of metric, which the withdrawn result did not. Figure 4.4 plots the four constraint outcomes of that table, each on its own scale, because they differ by three orders of magnitude and a shared axis would render three of the four flat.

The joint-limit row is the cleanest single result in the dataset. Joint-limit contact occurs in 10.70 % of baseline episodes and in none of either constrained arm’s 15,000 episodes, on any seed. Three qualifications belong with it. First, this is an observed zero over a finite sample and not a proof of impossibility; the honest statement is that no joint-limit contact was observed, with the sample size given so that a reader can bound the claim themselves. Second, the result is notable precisely because the joint-limit term had been classified as inactive by construction until the recalibration of Section 3.9 reclassified it, so the budget is being spent across more than the single manipulability term that earlier write-ups of this project

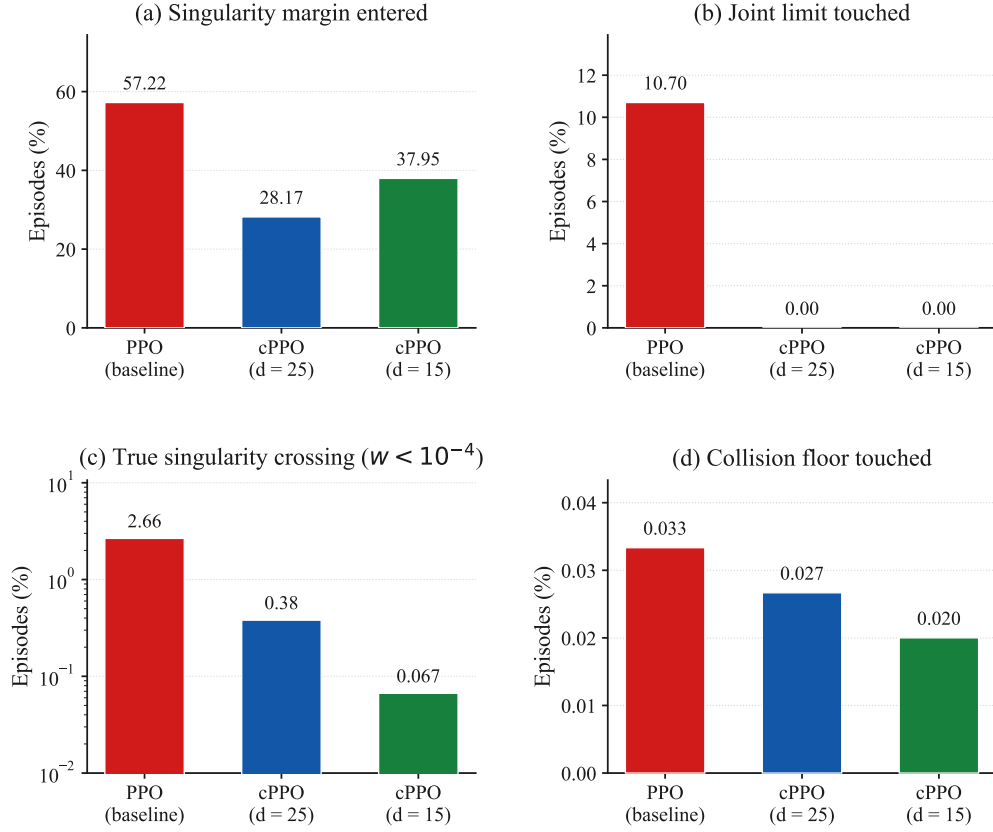


Figure 4.4. The four safety outcomes of Table 4.4, each panel independently scaled. Panel (c) uses a logarithmic axis. Note that (a) counts entry into the calibrated soft margin while (c) counts an actual loss of a degree of freedom, which is why the constrained arms improve far more on (c) than on (a): the constraint does not merely keep the arm out of the margin, it prevents the margin excursions from deepening into genuine singularities.

described. Third, the baseline’s 10.70 % is itself a seed lottery rather than a stable property: the per-seed rates are 47.83, 0.00, 0.00, 0.00 and 5.67 %, so a single seed contributes almost all of it. That pattern is the subject of Section 4.6.

Figure 4.5 separates the training-time cost into its three terms and shows where the budget is actually spent. The joint-limit panel is the clearest: the baseline’s contribution grows steadily from roughly iteration 700 onward while both constrained arms hold at zero for the whole run, which is the training-time counterpart of the evaluation row just discussed. The collision panel is flat at the vertical scale of a thousandth of a cost unit, confirming the term is inactive rather than merely small.

Collision was near zero for all three arms and remains so. It is reported for completeness and confirms that the collision floor is genuinely inactive at these settings.

One counter-result must be stated plainly rather than omitted because it is unflattering. The worst single-episode manipulability is identical for all three arms at 0.000001. Whatever the constraint does, it does not make the rare worst-case excursion shallower, and tightening the

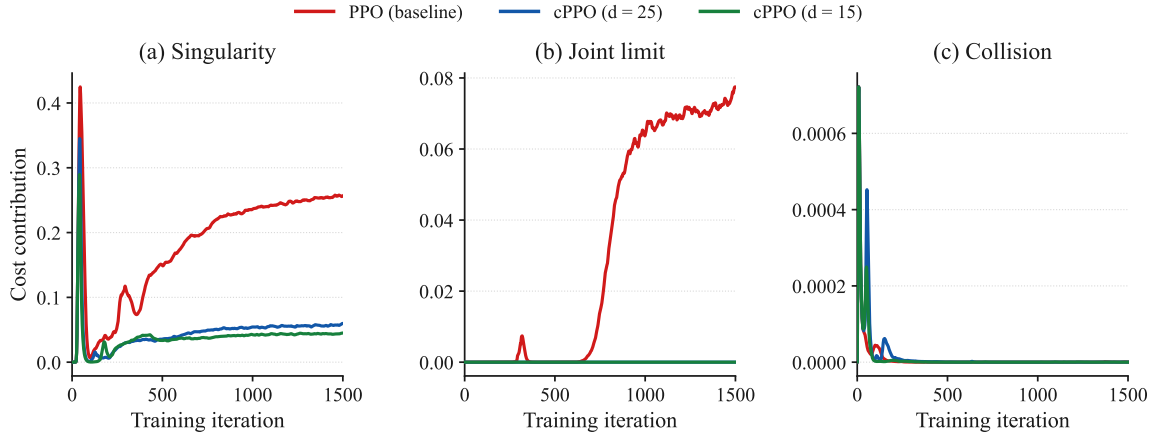


Figure 4.5. Episodic safety cost separated into its three terms, averaged across the five seeds. Note the differing vertical scales: the singularity term is an order of magnitude larger than the joint-limit term and two orders larger than collision. Curves are exponentially smoothed for legibility.

budget from 25 to 15 does not change that either. It reduces how often and how consistently the policy approaches a singularity, and the aggregate consequences of that are visible in the episodic-cost rows, where the worst episode costs 219.59 and 228.51 against the baseline’s 343.01 and the ninetieth percentile falls from 200.38 to 56.90 and 43.77. The worst *episode* is less costly overall even though the worst *instant* is equally deep. A claim that constrained reinforcement learning bounds the depth of the worst singular excursion is not supported by this data and is not made. For a safety argument the distinction matters practically: on hardware, a policy that visits a near-singular configuration one seventh as often but just as deeply when it does still requires the same instantaneous protection, and buys its margin in expected exposure rather than in worst-case severity. This is the distinction Brunke *et al.* [?] draw between constraint satisfaction in expectation and a safety certificate, discussed in Section 2.6.

The mechanism behind that distinction is visible in Figure 4.6. The mean episode-minimum manipulability of the $d = 15$ arm settles roughly four times higher than the baseline’s, so the typical episode operates further from the singular set. The worst instant, which no averaged curve can show, is unchanged at 0.000001 on all three arms.

Figure 4.7 closes the section with the quantity the budget is written against. The baseline’s episodic cost climbs throughout training and settles near 82, more than three times its own budget, because nothing in its objective penalises it for doing so. Both constrained arms settle beneath their budgets and stay there.

4.6 The principal finding: collapse of seed-to-seed variance

The largest effect measured in this batch is not visible in any mean. It appears only when the five seeds are examined individually, which Table 4.5 does. That seed-to-seed spread is

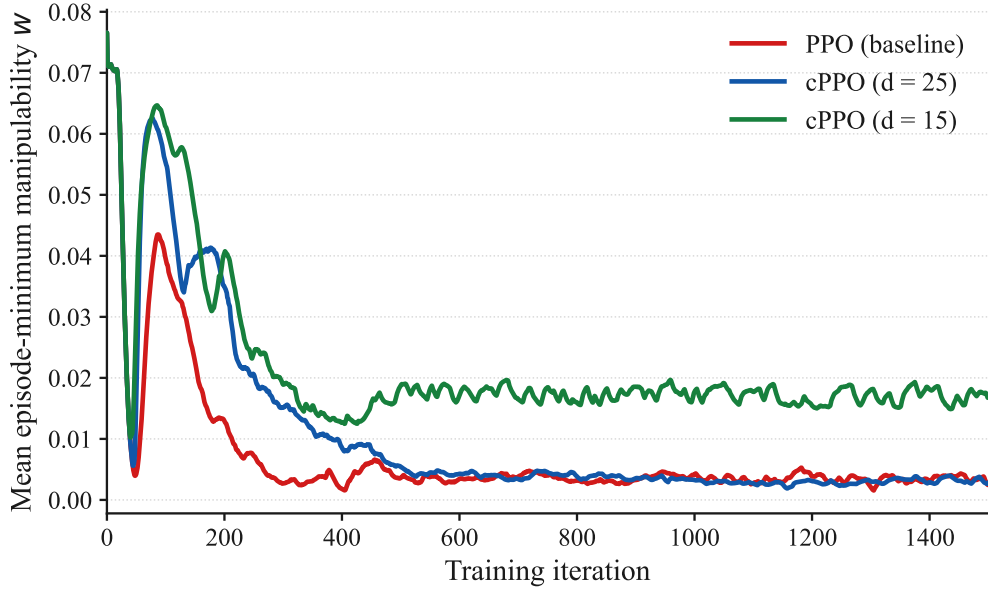


Figure 4.6. Mean episode-minimum manipulability w against training iteration, averaged across the five seeds. Higher is safer. The tighter budget holds the arm materially further from the singular set for the whole of the second half of training. Curves are exponentially smoothed for legibility.

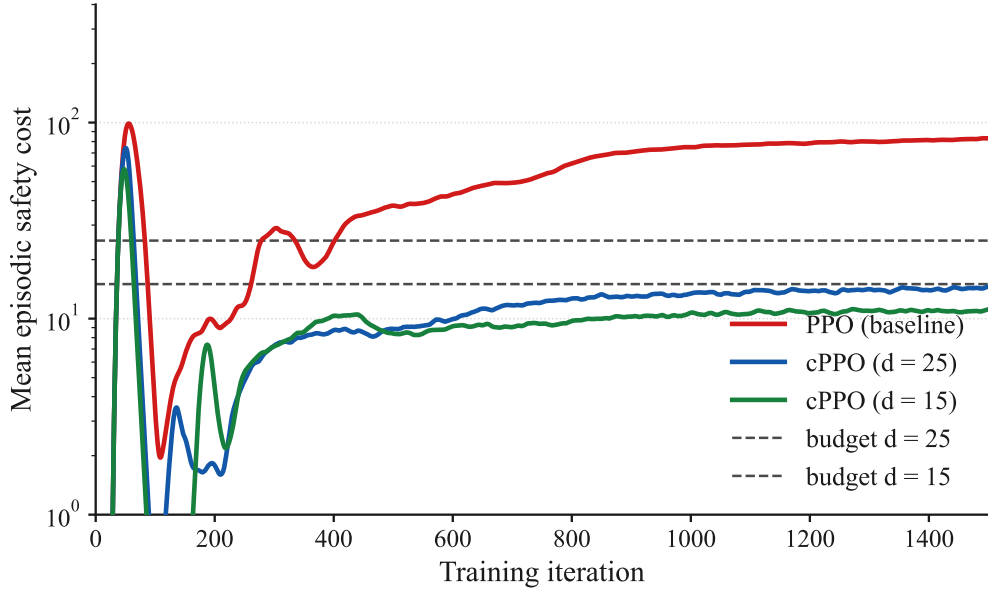


Figure 4.7. Mean episodic safety cost against training iteration, averaged across the five seeds, on a logarithmic axis. The dashed lines are the two budgets. The vertical axis is clipped below 1, which suppresses an early-training transient in the first two hundred iterations; the full range spans six decades and would compress the converged region into a sliver. Curves are exponentially smoothed for legibility.

itself a documented property of deep reinforcement learning rather than a peculiarity of this environment: Henderson *et al.* [?] show that random seed alone can produce statistically distinct outcome distributions for the same algorithm on the same task, and it is on that basis that this study reports dispersion across seeds rather than a single mean.

Table 4.5. Per-seed episodic cost, training tail mean and evaluation mean, with the peak Lagrange multiplier reached during training.

Quantity	s1	s3	s4	s52	s54	spread
<i>Training, tail-mean episodic cost</i>						
PPO (baseline)	102.10	162.30	30.04	106.93	7.95	20.4×
cPPO, $d = 25$	18.01	11.93	19.69	9.50	11.98	2.1×
cPPO, $d = 15$	14.08	11.61	3.83	10.65	14.53	3.8×
<i>Evaluation, mean episodic cost over 3000 episodes</i>						
PPO (baseline)	104.52	164.55	31.59	109.41	10.20	16.1×
cPPO, $d = 25$	20.20	12.04	21.39	8.91	13.50	2.4×
cPPO, $d = 15$	16.02	11.15	3.43	10.63	15.29	4.7×
<i>Peak λ reached during training</i>						
cPPO, $d = 25$	15.84	40.46	30.30	46.32	48.05	
cPPO, $d = 15$	17.62	35.47	40.83	27.13	38.56	

The same training data is plotted in Figure 4.8, where each seed contributes one vertical segment joining its three arm values. The figure is drawn this way, rather than as three sorted bands, because a sorted band would show the collapse in spread while hiding which direction each seed moved in. Both facts matter to the reading below.

Figure 4.9 shows the same collapse developing over training rather than at its endpoint, with every seed drawn separately. The baseline panel is the one to look at: five runs of the same algorithm in the same environment, differing only in the random seed, ending anywhere between 8 and 162. The constrained panels are not merely lower, they are narrow.

The baseline row is the behaviour of unconstrained policy optimisation with the cost merely observed and never acted upon, and it is extraordinarily inconsistent. Its natural episodic cost ranges from 7.95 on seed 54 to 162.30 on seed 3, a spread of more than twentyfold, for the identical algorithm in the identical environment differing only in the random seed. Seed 54 produces a policy that is accidentally safe. Seeds 3, 52 and 1 produce policies that are severely unsafe. Nothing in the training procedure distinguishes them and nothing in the training telemetry would warn an engineer which one they had obtained. The same lottery is visible in the evaluation column, and in the joint-limit rates of Section 4.5, where one seed touches a joint limit on 47.83 % of its episodes and three seeds never touch one at all.

Both constrained arms pull every seed into a narrow band beneath their budget irrespective

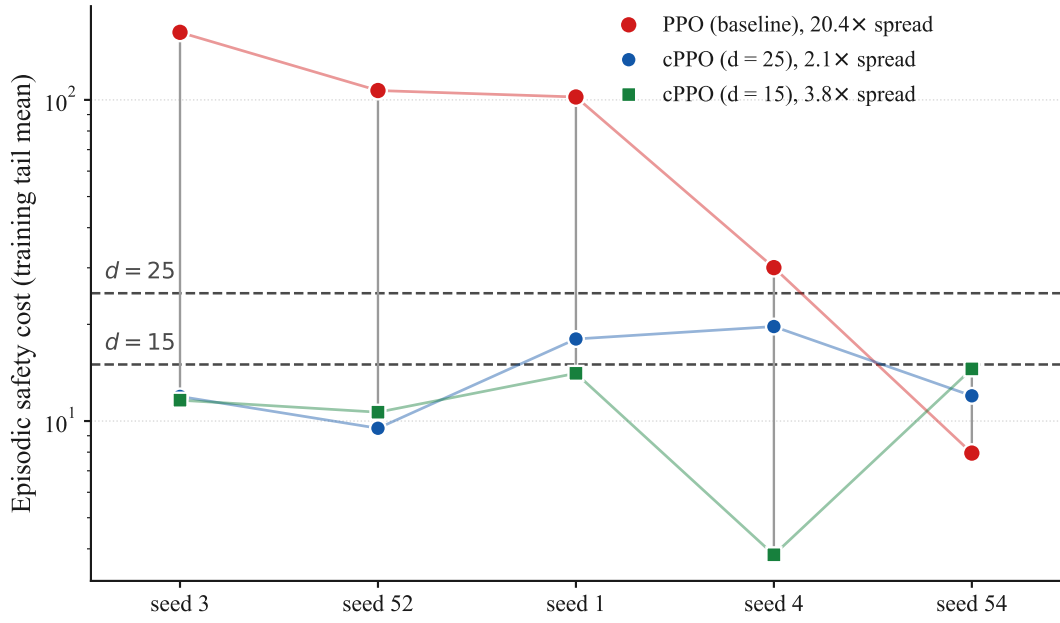


Figure 4.8. Per-seed episodic safety cost on a logarithmic scale. Each vertical segment joins the three arms for one seed, and the two horizontal lines are the budgets. Seeds are ordered by descending baseline cost rather than by seed number, so that the baseline’s twentyfold sweep across the plot can be compared against the two constrained arms, which stay flat. The spread beside each legend entry is the ratio of that arm’s highest seed to its lowest.

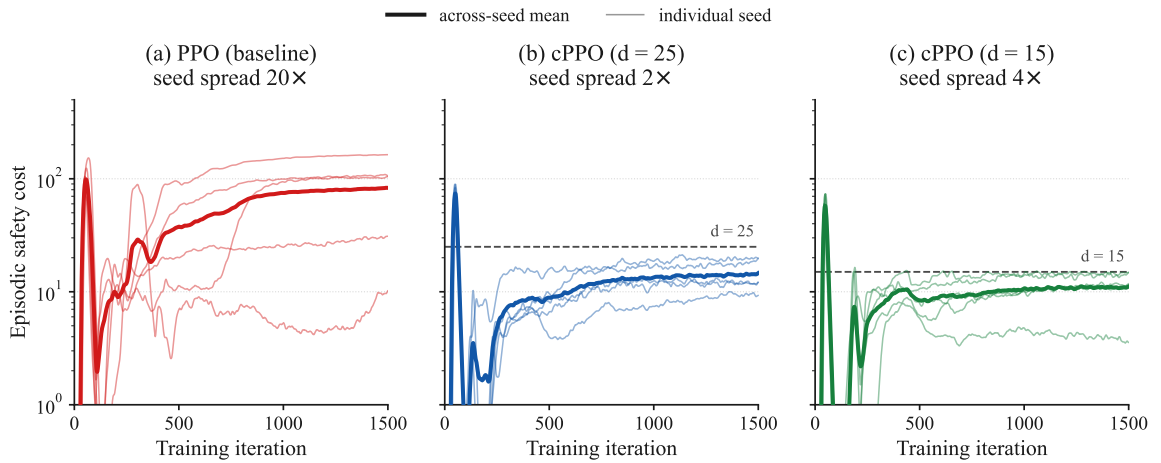


Figure 4.9. Training-time episodic cost for every seed, one panel per arm, on a shared logarithmic scale. Thin lines are individual seeds and the thick line is their mean. The spread quoted above each panel is the ratio of the highest to the lowest seed at the end of training. Curves are exponentially smoothed for legibility.

of where its unconstrained counterpart sat. Seed 3, whose baseline counterpart is the worst in the batch at 162.30, ends at 11.93 and 11.61.

This should not be read as a uniform improvement, and the per-seed data makes the qualification unavoidable, though it is a narrower qualification than the previous version of this chapter reported. The band is entered from both directions, but on one seed of five rather than on most of them. On seed 54, whose unconstrained policy was already comfortably under budget at 7.95, both constrained arms finish *higher*, at 11.98 and 14.53 in training and 13.50 and 15.29 in evaluation. On the other four seeds the cost falls sharply. What the constraint supplies is therefore best described as a ceiling rather than a reduction applied to every run: it prevents the disastrous outcomes without guaranteeing that a fortunate run stays as fortunate as it would have been. Since which of the two a given training run will produce is not knowable in advance, which is precisely the lottery described above, trading an unpredictable draw between 7.95 and 162.30 for a reliable band around 12 is the correct trade for a system intended to run on hardware. It remains a trade, and it is presented as one.

The effect is confirmed independently on held-out evaluation episodes rather than on training rollouts. Mean episodic cost falls from 84.05 to 15.21 and 11.31, reductions of 81.9 % and 86.5 %. The more important quantity is the standard deviation across seeds, which falls from 62.75 to 5.38 and 5.01, a tightening of 11.7 and 12.5 times, and a larger proportional change than the change in the mean. The baseline’s standard deviation of 62.75 is itself smaller than its own mean of 84.05 only narrowly, which is the formal signature of the lottery described above.

The engineering reading is that unconstrained optimisation on this task does not have a safety level. It has a distribution of safety levels, and which one is drawn is close to arbitrary. The constrained agent’s contribution in this batch is to make the outcome predictable, at no measurable task cost. For a manipulator intended to run on real hardware this is arguably the more useful property of the two: a mean improvement tells an integrator what to expect on average across training runs they will never perform, whereas a variance collapse tells them what to expect from the single policy they actually trained. A safety argument that depends on having drawn a fortunate seed is not a safety argument.

4.7 The multiplier, measured rather than inferred

The previous version of this chapter recorded only the value of λ at the final iteration and inferred the rest of its behaviour. The full per-iteration trajectories have since been extracted from the training logs, and Figure 4.10 plots them. The inference is confirmed and can now be stated as a measurement.

The pattern is the same on every seed and at both budgets. The multiplier sits at zero for roughly the first forty iterations while the policy is still learning to reach at all and its episodic

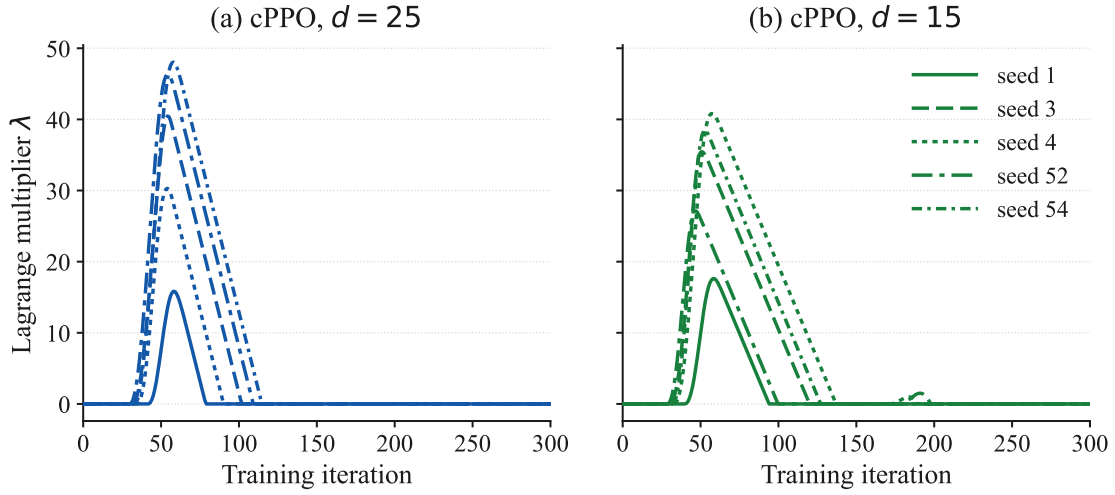


Figure 4.10. Lagrange multiplier against training iteration, all five seeds, both constrained arms. The multiplier is zero for the remaining 1200 iterations in every run.

cost is low for want of competence. It then rises steeply as the policy becomes capable enough to exceed the budget, reaching a single sharp peak between iterations 47 and 58, and decays back to zero once the cost has been driven under the budget. Peak values run from 15.84 to 48.05 at $d = 25$ and from 17.62 to 40.83 at $d = 15$, and λ is above 0.01 for between 36 and 120 iterations of the 1500. Nine of the ten runs end with the multiplier at exactly zero; the tenth, seed 1 at $d = 15$, ends at 0.053.

Two things follow. The first is that a final λ of zero says nothing about whether the constraint was active, which is why the previous version’s table of final values was close to uninformative. On every seed the constraint engaged hard and then relaxed, and reading the final value alone would have suggested that it never engaged at all. The second is that the shape is the one the optimiser’s documented dynamics predict rather than an interpretation invented to fit these numbers. Plain dual ascent on the multiplier is integral control on the constraint, and an integral controller with no proportional or derivative action overshoots and then unwinds, which is precisely the behaviour Stooke *et al.* [?] analyse and the reason they propose adding proportional and derivative terms. Section 2.5 sets this out, and Chapter 6 identifies the PID form as the natural extension of this work. The single clean overshoot visible in Figure 4.10, rather than the sustained oscillation of their Figure 1, reflects that the cost here is driven under budget and stays there, so the controller has nothing further to correct.

4.8 The effect of tightening the budget

The two constrained arms differ in one number, so the comparison between them isolates the effect of how hard the constraint is applied. Section 2.11 identifies this as a gap in the published work, where the budget of 25 is inherited and never varied.

Tightening from 25 to 15 improves nearly every safety measure and costs nothing measurable. Singularity crossings fall further, from 57 episodes to 10, so the tighter budget removes another 82 % of what the looser one left. Mean episodic cost falls from 15.21 to 11.31 and the ninetieth percentile from 56.90 to 43.77. Joint-limit contact was already at zero and stays there. On the task side the tighter arm is marginally the best of the three at every goal-reach threshold and carries the highest training reward at 134.09, so no trade-off against task performance is visible at this budget.

Three qualifications keep that from being read as a general rule. The tighter budget does not improve the soft-margin singularity fraction during training, which rises from 0.169 to 0.233, and the two measures disagree because one is taken on an exploring policy against a calibrated margin and the other on a frozen policy against a true rank deficiency. It does not improve the worst single-instant manipulability, which is unchanged. And the seed-to-seed spread of episodic cost is slightly wider at $d = 15$ than at $d = 25$ in relative terms, at 4.7 against 2.4 times, because the tighter budget drives one seed down to 3.43 while another sits at 15.29. The defensible statement is that within the range tested, tightening the budget continued to help and did not begin to cost task performance. Whether that continues below 15 is untested, and Section 4.9 records it as such.

4.9 Limitations

Four limitations bound what this chapter establishes, and they are stated here in full rather than distributed as caveats.

The first is the seed count. Five seeds is enough to demonstrate that the outcome of unconstrained training is a lottery and that the constraint closes the spread, and it is consistent with the practice Henderson *et al.* [?] describe as a minimum. It is not enough to put a confidence interval on the size of the effect. Every dispersion figure in this chapter should be read as an estimate from five draws.

The second is that no off-policy arm was trained. The *sac* arm registered in the original plan is absent, so the algorithm-family question examined in comparable work on manipulation tasks [?] remains open in this setting. What this batch does answer, and what the previous version of this chapter listed as its principal missing piece, is the effect of an actively binding budget: at $d = 15$, which is below the baseline’s natural operating cost on every seed, the constraint binds throughout and Section 4.8 reports the result.

The third is a pair of data-hygiene faults found during analysis, filtered around, and not repaired at source. Three superseded pre-audit constrained runs from the gradient-clip-bug era still occupy the same experiment directory as the current runs under identical labels; the results reported here exclude them by checkpoint-path date, and the evaluation script’s checkpoint selection was verified to resolve to the newer run, but both protections rest on

file metadata rather than on the stale runs having been removed. Separately, the append-only evaluation results file was found to carry stale rows from a superseded evaluation sweep, which were filtered by checkpoint path before analysis. Neither fault affected the numbers reported here, since both were caught and excluded, but both are standing risks to any future automated aggregation. They are documented rather than quietly fixed because the discipline of recording near-misses is part of this project’s method, and because the entire correction that produced this chapter began as exactly such a near-miss that was not caught in time.

The fourth is the counter-note of Section 4.5, restated because it is the sharpest limit on the safety claim itself. The worst single-episode manipulability is identical across all three arms. The constraint demonstrably reduces the frequency and the seed-to-seed consistency of near-singular operation, and it reduces the cost of the worst episode, but it does not reduce the depth of the worst instant, and tightening the budget does not either. Any hardware safety case built on this result must rest on expected exposure rather than on worst-case severity, and must retain whatever instantaneous protection it would otherwise have required.

4.10 Summary

Four claims are established by this batch.

First, the control arm reproduces the unconstrained baseline exactly, to identical stored weights, on all five seeds, and again independently across all 15,000 evaluation episodes. That confirms the gradient-clipping correction and licenses the constrained-versus-baseline comparison to be read directly, because the artifact term of Equation (4.1) is exactly zero rather than merely small.

Second, the constraint costs no measurable task performance at either budget. Reward differs by less than the seed-to-seed standard deviation and the tighter arm ends marginally above the baseline; goal-reach is equal or better at every threshold and across the whole distance distribution.

Third, on safety, the constraint reduces true singularity crossings by a factor of 7.0 at a budget of 25 and 39.9 at a budget of 15, and eliminates all observed joint-limit contact at both. Its largest measured effect, however, is on variance rather than on the mean. It collapses a twentyfold seed-to-seed spread in natural episodic cost to roughly two, and a standard deviation of 62.75 across seeds to about 5, converting an outcome that was close to a lottery into a predictable one. The same collapse appears unexpectedly in task precision, where the spread of sub-centimetre success falls from 7.52 points to about 1.

Fourth, the multiplier trajectories show a single sharp engagement peaking near iteration 50 and decaying to zero, on every seed and at both budgets, which is the behaviour plain dual ascent is known to produce and confirms by measurement what the previous version of this

chapter could only infer.

The comparison this project registered in advance as its central claim, an actively binding budget against the control arm, is answered here by the $d = 15$ arm, and tightening the budget was found to help further at no task cost within the range tested. What is not answered is the off-policy comparison, and what remains open is whether the benefit continues below a budget of 15. Alongside those findings sits a methodological one: a previously reported effect of this kind in this project was an implementation artifact, the artifact has been removed, its removal has been verified as strongly as such a thing can be verified, and the corrected comparison still shows a substantial and differently shaped safety benefit. Reporting that sequence honestly is a more defensible contribution than the original headline it replaces [?].